

SmartBancs App - Ejecución, instalación y puesta en marcha

La solución puede ejecutarse localmente mediante **Docker Compose**, evitando la necesidad de instalar PostgreSQL, Redis o Prometheus directamente en el sistema operativo. El repositorio contiene el código fuente, configuración, scripts, infraestructura y archivos necesarios para reproducir el entorno del MVP.

El código fuente se encuentra disponible en:

Repositorio: <https://github.com/andrea-as/smartbancs-mvp>

El procedimiento descrito a continuación parte de un equipo limpio con Windows, Docker Desktop y Git instalados.

1.1. Prerrequisitos

Antes de iniciar se requiere:

Requisito	Uso
Git	Clonar el repositorio
Docker Desktop	Ejecutar los contenedores
Docker Compose	Orquestar los servicios del MVP
Python 3.x	Ejecutar el ETL de forma local
Navegador web	Acceder al dashboard, Swagger y Prometheus

Docker Compose se encuentra integrado en las versiones actuales de Docker Desktop, por lo que no es necesario instalarlo como componente independiente.

Puertos utilizados

El entorno utiliza los siguientes puertos locales:

Puerto	Servicio	Uso
8000	SmartBancs API	API REST + Dashboard

8001	AI Service	Servicio de IA
5432	PostgreSQL	Base de datos
6379	Redis	Cola de trabajos
9090	Prometheus	Métricas

Antes de ejecutar el proyecto se debe verificar que estos puertos no estén siendo utilizados por otro servicio.

1.2. Verificar prerequisites

Abrir **PowerShell**, CMD o una terminal de Git Bash.

Verificar Git

```
git --version
```

Debe mostrarse una versión instalada de Git, por ejemplo:

```
git version 2.x.x
```

Verificar Docker

```
docker --version
```

Verificar Docker Compose

```
docker compose version
```

Se espera una salida similar a:

```
Docker Compose version v2.x.x
```

Verificar Python

Python solamente es necesario para ejecutar directamente el script ETL fuera del contenedor:

```
python --version
```

16.3. Clonar el repositorio

El primer paso consiste en descargar el repositorio desde GitHub.

Git permite clonar una copia completa del repositorio remoto en el equipo local. El procedimiento oficial consiste en utilizar git clone seguido de la URL del repositorio.

Primero se debe seleccionar la carpeta donde se almacenará el proyecto.

Por ejemplo, en Windows:

```
cd D:\TATA
```

Si la carpeta no existe:

```
mkdir D:\TATA
```

```
cd D:\TATA
```

Posteriormente se ejecuta:

```
git clone https://github.com/andrea-as/smartbancs-mvp.git
```

Git descargará el proyecto y creará automáticamente una carpeta llamada:

```
smartbancs-mvp
```

Por lo tanto, la estructura local será:

```
D:\TATA\
```

```
└── smartbancs-mvp\
```

1.4. Ingresar al proyecto

Después de realizar la clonación:

```
cd smartbancs-mvp
```

Se puede verificar que se está ubicado correctamente mediante:

```
Get-Location
```

También se puede listar el contenido:

dir

Se deberían observar elementos similares a:

.github

ansible

db

docs

etl

evidence

k8s

observability

scripts

services

sqlserver

terraform

tests

docker-compose.yml

README.md

La estructura corresponde a los componentes entregados en el repositorio.

1.5. Verificar el estado del repositorio

Opcionalmente, se puede comprobar que Git reconoce correctamente el proyecto:

git status

Para consultar la rama actual:

git branch

Para consultar el repositorio remoto:

```
git remote -v
```

La salida debe mostrar el repositorio:

```
https://github.com/andrea-as/smartbancs-mvp.git
```

1.6. Construcción y levantamiento del entorno

Desde la carpeta raíz del proyecto:

```
docker compose up --build -d
```

¿Qué realiza este comando?

El comando:

```
docker compose up --build -d
```

realiza tres acciones principales:

1. **docker compose**

Utiliza la configuración definida en `docker-compose.yml`.

2. **--build**

Construye nuevamente las imágenes necesarias a partir de los Dockerfiles del proyecto.

3. **-d**

Ejecuta los contenedores en segundo plano (*detached mode*), permitiendo continuar utilizando la terminal.

Durante el primer arranque Docker puede tardar más tiempo porque debe descargar las imágenes base, instalar dependencias y construir los servicios.

1.7. Verificar los contenedores

Una vez finalizada la construcción:

```
docker compose ps
```

Los servicios principales deberán aparecer en estado running o equivalente.

Para consultar los contenedores activos:

```
docker ps
```

La solución utiliza, entre otros componentes:

- API SmartBancs;
- PostgreSQL;
- Redis;
- servicio de IA;
- Prometheus.

El README del repositorio especifica que el MVP se ejecuta completamente en local y no requiere cuentas de nube ni servicios de pago.

1.8. Revisar los logs

Para observar los logs de todos los servicios:

```
docker compose logs
```

Para seguir los logs en tiempo real:

```
docker compose logs -f
```

Para consultar únicamente los logs de la API:

```
docker compose logs -f api
```

Para consultar los logs del servicio de IA:

```
docker compose logs -f ai
```

Para salir de la visualización de logs:

```
Ctrl + C
```

Esto no detiene los contenedores; solamente termina la visualización de los logs.

1.9. Verificación de salud de la API

La API expone un endpoint de health check.

Desde PowerShell:

```
curl.exe http://localhost:8000/health
```

Una respuesta correcta indica que el servicio está disponible.

También se puede abrir directamente en el navegador:

```
http://localhost:8000/health
```

1.10. Interfaces disponibles

Una vez levantado el entorno, se dispone de las siguientes interfaces.

Dashboard

```
http://localhost:8000/
```

El dashboard permite visualizar el estado general del MVP, cuentas demo, operaciones y métricas principales.

Swagger / OpenAPI

```
http://localhost:8000/docs
```

Swagger permite consultar y ejecutar los endpoints REST directamente desde el navegador.

Métricas Prometheus de la API

```
http://localhost:8000/metrics
```

Este endpoint devuelve las métricas en formato compatible con Prometheus.

Prometheus

```
http://localhost:9090
```

Prometheus permite consultar las métricas recolectadas por la plataforma.

Servicio de IA

<http://localhost:8001/health>

Permite comprobar independientemente la disponibilidad del servicio de IA.

Journey sintético

<http://localhost:8000/synthetic/transfer>

Permite ejecutar una verificación sintética del flujo de transferencia y utilizarla como indicador de disponibilidad y experiencia digital.

Los endpoints anteriores forman parte de la configuración documentada actualmente en el repositorio.

1.11. Probar una transferencia

Una vez que todos los servicios están activos, se puede ejecutar una transferencia.

Transferencia

Prueba una operación financiera con bloqueo transaccional.

Cuenta origen

Cuenta destino

Monto (USD)

Ejecutar transferencia

```
{  "transaction_id": "618775aa-c9df-4c29-992b-040e646868fa",  "status": "COMPLETED"}
```

El endpoint procesa la transferencia mediante una transacción ACID y bloquea las cuentas involucradas en un orden determinista.

Una respuesta exitosa debe indicar que la transferencia fue completada.

1.12. Consultar el saldo después de la transferencia

Para consultar la cuenta origen:

```
curl.exe http://localhost:8000/api/v1/accounts/ACC-1004
```

Para consultar la cuenta destino:

```
curl.exe http://localhost:8000/api/v1/accounts/ACC-1003
```

```
PS C:\Users\Andrea> curl.exe http://localhost:8000/api/v1/accounts/ACC-1004
{"account_id":"ACC-1004","owner_id":"CUS-004","currency":"USD","balance":1119.42,"version":1}
PS C:\Users\Andrea> curl.exe http://localhost:8000/api/v1/accounts/ACC-1003
{"account_id":"ACC-1003","owner_id":"CUS-003","currency":"USD","balance":830.58,"version":1}
PS C:\Users\Andrea> |
```

Esto permite comprobar que el débito y crédito fueron aplicados correctamente.

Así mismo, se puede consultar el saldo de cada transacción desde la Interfaz. Para ello, se debe dar clic en el botón “Actualizar saldos”.

Cuentas demo

Selecciona una cuenta para consultar su información. Son datos locales del MVP; no se muestran contraseñas ni datos reales.

Actualizar saldosOcultar saldos10 cuentas disponibles

Cuenta a consultar

ACC-1001

Cuenta	Cliente demo	Moneda	Saldo actual	Versión
ACC-1001	CUS-001	USD	1000.00 USD	0

Versión = contador interno de cambios de saldo de la cuenta; ayuda a demostrar concurrencia y control de actualizaciones.

Por consiguiente, seleccionamos la cuenta a consultar para poder visualizar la respectiva información:

Cuentas demo

Selecciona una cuenta para consultar su información. Son datos locales del MVP; no se muestran contraseñas ni datos reales.

Actualizar saldos

Ocultar saldos

10 cuentas disponibles

Cuenta a consultar

ACC-1001

ACC-1001

ACC-1002

ACC-1003

ACC-1004

ACC-1005

ACC-1006

ACC-1007

ACC-1008

ACC-1009

ACC-1010

Cuentas demo

Selecciona una cuenta para consultar su información. Son datos locales del MVP; no se muestran contraseñas ni datos reales.

Actualizar saldos

Ocultar saldos

10 cuentas disponibles

Cuenta a consultar

ACC-1004

Cuenta	Cliente demo	Moneda	Saldo actual	Versión
ACC-1004	CUS-004	USD	1119.42 USD	1

Versión = contador interno de cambios de saldo de la cuenta; ayuda a demostrar concurrencia y control de actualizaciones.

1.13. Consultar la recomendación generada por IA

La transferencia genera un evento que es procesado de manera asíncrona mediante Redis y el servicio de IA.

Para consultar las recomendaciones asociadas a la cuenta:

curl.exe http://localhost:8000/api/v1/recommendations/ACC-1001

```
PS C:\Users\Andrea> curl.exe http://localhost:8000/api/v1/recommendations/ACC-1001
[]
PS C:\Users\Andrea> curl.exe http://localhost:8000/api/v1/recommendations/ACC-1004
[{"account_id":"ACC-1004","recommendation":"Mantener seguimiento de gastos pequeños para detectar patrones.","model_version":"demo-1.0","created_at":"2026-09-21T00:28:47.634735+00:00"}]
```

Recomendaciones de IA

La recomendación se procesa en segundo plano y no bloquea la transferencia.

Cuenta

ACC-1004

Consultar recomendaciones

```
{
  "ultima_recomendacion": {
    "account_id": "ACC-1004",
    "recommendation": "Mantener
seguimiento de gastos pequeños para
detectar patrones.",
    "model_version": "demo-1.0",
    "created_at": "2026-09-
21T00:28:47.634735+00:00"
  },
  "recomendaciones_disponibles": 1
}
```

La transferencia no necesita esperar a que finalice este procesamiento para ser confirmada.

1.14. Probar la integración con Bancs

El MVP incluye un endpoint para representar la recepción de eventos provenientes del core legado.

Integración con Bancs

Simula la recepción y normalización de un evento del Core legado.

Cuenta origen del evento

Cuenta destino del evento

Monto informado por Bancs (USD)

Enviar evento Bancs de prueba

```
{
  "status": "ACCEPTED",
  "legacy_id": "UI-1789950887841"
}
```

El adaptador realiza tareas como:

- limpieza de identificadores;
- normalización de moneda;
- conversión del monto;

- normalización de fecha;
- generación del evento canónico;
- registro para auditoría.

El objetivo del endpoint en el MVP es demostrar la transformación y recepción desacoplada de eventos del sistema legado; no representa una modificación directa del core Bancs.

Esto lo podemos comprobar desde el powershell ejecutando el comando:

`docker compose exec -T db psql -U smartbancs -d smartbancs -c "SELECT id, details, created_at FROM audit_events WHERE event_type = 'BANCS_EVENT_NORMALIZED' ORDER BY id DESC LIMIT 1;"`

```
PS D:\CLONARTATA\smartbancs-mvp> docker compose exec -T db psql -U smartbancs -d smartbancs -c "SELECT id, details, created_at FROM audit_events WHERE event_type = 'BANCS_EVENT_NORMALIZED' ORDER BY id DESC LIMIT 1;"
id |
    details |
-----+-----
2 | {"amount": "25.50", "currency": "USD", "legacy_id": "UI-1789950887841", "event_time": "2026-09-21T00:34:47.841000+00:00", "request_id": "719f40c6-f395-41af-af94-1724ae546faf", "source_system": "Bancs", "source_account": "ACC-1004", "destination_account": "ACC-1002"} | 2026-09-21 00:34:47.85635+00
(1 row)
```

1.15. Ejecutar el proceso ETL

El ETL puede ejecutarse directamente desde el equipo local.

Primero comprobar que Python está disponible:

`python --version`

Instalar la dependencia requerida:

`pip install pandas`

Posteriormente ejecutar:

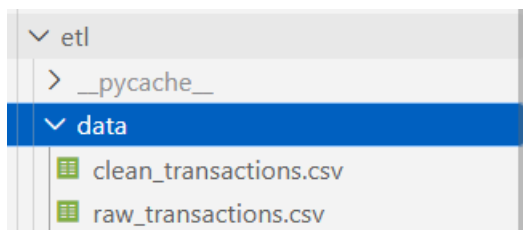
`python etl\transform_transactions.py`

El proceso toma el lote de transacciones de prueba y realiza:

- limpieza de valores nulos;

- normalización de fechas;
- normalización de montos;
- estandarización de texto;
- tratamiento de duplicados;
- validaciones de calidad;
- generación de indicadores.

Dentro de la carpeta correspondiente se encuentran dos archivos clave:



- raw_transactions.csv, que contiene los datos originales sin procesar.

```

1 transaction_id,account_id,amount,currency,event_time,status
2 tx-001, acc-1001 ,100.50,usd,2026-09-15 10:00:00,completed
3 tx-002,acc-1002,,USD,2026-09-15 10:01:00,completed
4 tx-003,ACC-1003,250.00,usd,2026-09-15T10:02:00Z,completed
5 tx-003,ACC-1003,250.00,usd,2026-09-15T10:02:00Z,completed
6 tx-004,ACC-1004,abc,USD,invalid_date,completed
7

```

- clean_transactions.csv, generado tras la ejecución del proceso de **ETL**, en el cual se aplicaron las transformaciones necesarias para depurar, normalizar y estructurar la información.

```

1 transaction_id,account_id,amount,currency,event_time,event_date,status,quality_amount_ok,quality_date_ok,quality_id_ok
2 tx-001,acc-1001,100.5,USD,2026-09-15 10:00:00+00:00,2026-09-15,COMPLETED,True,True,True
3 tx-002,acc-1002,,USD,2026-09-15 10:01:00+00:00,2026-09-15,COMPLETED,False,True,True
4 tx-003,ACC-1003,250.0,USD,,COMPLETED,True,False,True
5 tx-003,ACC-1003,250.0,USD,,COMPLETED,True,False,True
6 tx-004,ACC-1004,,USD,,COMPLETED,False,False,True
7

```

Este contraste evidencia el valor del flujo de ETL: convertir datos crudos en insumos confiables y listos para análisis o integración con otros sistemas.

1.16. Ejecutar pruebas de carga

El repositorio contiene un script de pruebas de carga en:

```
tests/load_test.py
```

Para ejecutar una prueba controlada:

```
$env:TOTAL_REQUESTS="100"
```

```
$env:CONCURRENCY="20"
```

```
python tests/load_test.py
```

Para reproducir el escenario de 10.000 solicitudes con concurrencia 20:

```
$env:TOTAL_REQUESTS="10000"
```

```
$env:CONCURRENCY="20"
```

```
python tests/load_test.py
```

Para realizar la prueba agresiva:

```
$env:TOTAL_REQUESTS="10000"
```

```
$env:CONCURRENCY="1000"
```

```
python tests/load_test.py
```

Los resultados deben interpretarse como **benchmarks del entorno local**, no como certificación de capacidad productiva de 10.000 TPS.

El propio repositorio establece esta distinción y reporta los resultados medidos en el equipo de desarrollo.

1.17. Revisar las métricas

Las métricas de la API pueden visualizarse directamente mediante:

```
http://localhost:8000/metrics
```

También pueden consultarse desde Prometheus:

http://localhost:9090

Las métricas permiten analizar, entre otros aspectos:

- número de solicitudes;
- latencia;
- transferencias;
- errores de base de datos;
- trabajos de IA;
- errores de publicación en Redis;
- profundidad de procesamiento;
- ejecución del journey sintético.

Así mismo, podemos analizar las métricas más fundamentales desde la interfaz. Para ello, en la sección “Resumen de operación” damos clic en el botón de “Actualizar métricas”:

Resumen de operación

Resumen compacto de la actividad de API e IA. Prometheus conserva el detalle técnico.

Actualizar métricasActualizado: 7:48:38 p.m.

Solicitudes HTTP 1	Transferencias completadas 3872
Eventos Bancs aceptados 1	Trabajos IA completados 232
Errores de base de datos 0	Trabajos pendientes IA 3639

Formato original: [métricas técnicas](#).

1.18. Detener la solución

Para detener todos los servicios creados mediante Docker Compose:

```
docker compose down
```

Esto detiene y elimina los contenedores asociados al proyecto, pero conserva las imágenes construidas.

Si posteriormente se desea iniciar nuevamente el entorno:

```
docker compose up -d
```

No es necesario utilizar `--build` nuevamente mientras no hayan cambiado los Dockerfiles o dependencias de construcción.

1.19. Detener y eliminar también los volúmenes

Si se requiere realizar una ejecución completamente limpia, eliminando también los volúmenes asociados:

```
docker compose down -v
```

Advertencia: esta opción elimina los volúmenes de Docker asociados al proyecto y, por tanto, los datos persistidos de PostgreSQL utilizados por el entorno local.

Después de ejecutar este comando, el entorno puede reconstruirse nuevamente mediante:

```
docker compose up --build -d
```

1.20. Problemas frecuentes

Puerto ocupado

Si Docker informa que un puerto ya está en uso, se debe identificar el proceso que lo utiliza.

Por ejemplo, para el puerto 8000:

```
netstat -ano | findstr :8000
```

Para el puerto 5432:

```
netstat -ano | findstr :5432
```

Para el puerto 6379:

```
netstat -ano | findstr :6379
```

Para el puerto 9090:

```
netstat -ano | findstr :9090
```

Un contenedor no inicia

Consultar los servicios:

```
docker compose ps
```

Y posteriormente revisar los logs:

```
docker compose logs
```

Para un servicio específico:

```
docker compose logs api
```

o:

```
docker compose logs ai
```

Se realizaron cambios en el código y no aparecen

Reconstruir las imágenes:

```
docker compose down
```

```
docker compose up --build -d
```

La API todavía no responde después del arranque

Esperar unos segundos para permitir que PostgreSQL y Redis terminen su inicialización y volver a comprobar:

```
docker compose ps
```

Después:

curl.exe http://localhost:8000/health

Si continúa fallando:

docker compose logs api

2. Declaración de uso de IA

Durante la preparación del reto se utilizó un asistente de inteligencia artificial para apoyar actividades de desarrollo y documentación.

La IA se utilizó para:

- proponer y revisar la estructura del MVP;
- apoyar la redacción y revisión de documentación;
- sugerir casos de prueba;
- ayudar a diagnosticar errores de configuración;
- revisar cobertura de requisitos;
- apoyar componentes de API, ETL, observabilidad, infraestructura y documentación.

La revisión técnica, decisiones, ejecución local, validación y responsabilidad final de la solución corresponden al candidato.

La IA de negocio del MVP es determinista y demostrativa; no utiliza datos personales reales ni una API externa de pago.

3. Estructura del repositorio

